

Mini Assessment

APIs, JSON Parsing & Defensive Programming

Name	P Venkata Harshavardhan Reddy
Register No.	99230040739
Department	CSE

Q1. If an API returns 429, what should your program do?

A 429 status code means “Too Many Requests” — the client has exceeded the API's rate limit. The program should not simply ignore this or retry immediately in a tight loop, since that will keep failing and may get the client blocked entirely.

Correct handling:

- Check for a **Retry-After** header in the response and wait at least that long before retrying.
- If no Retry-After header is present, back off using an **exponential backoff** strategy (e.g. 1s, 2s, 4s, 8s...) instead of a fixed short delay.
- Cap the number of retry attempts so the program fails gracefully instead of retrying forever.
- Log the rate-limit event so it's visible for debugging and capacity planning.

Q2. If response.json() fails, what should your program do?

response.json() fails (raises an exception, typically a **JSONDecodeError**) when the response body is not valid JSON — for example, the server returned an HTML error page, an empty body, or plain text instead of JSON. Letting this exception crash the program is poor practice.

Correct handling:

- Wrap the call in a **try/except** block and catch the JSON decoding error specifically.
- Before parsing, check **response.status_code** and the **Content-Type** header to detect an error response early.
- On failure, log or print the raw **response.text** so the actual problem is visible for debugging.
- Return a safe fallback value or raise a clear, application-specific error rather than letting the raw exception propagate.

Q3. If a JSON key is missing, why might .get() be safer than []?

Accessing a dictionary with square brackets (e.g. **data["key"]**) raises a **KeyError** and crashes the program if the key does not exist. Using **data.get("key")** instead returns **None** (or a default value you specify) when the key is missing, so the program keeps running.

Why this matters:

- **.get("key", default)** lets you supply a sensible fallback value instead of crashing.

- APIs frequently omit optional fields or change their response shape over time — `.get()` tolerates this gracefully.
- It avoids the need to wrap every field access in its own `try/except` just to guard against a missing key.